

Computational Requirements

1. Purpose

This document lists the computational requirements to reproduce the data processing, model training, and mapping pipeline used in this project (rainfall raster processing, class-imbalance handling, machine learning model training/tuning, and hazard/vulnerability/risk mapping).

2. Hardware Requirements

2.1 Reference configuration (used to generate published results)

Component	Specification
Machine	HP Pavilion 15 laptop
Processor	Intel Core i5
RAM	16 GB DDR4, 3200 MHz
Storage	NVMe SSD
Operating System	Windows 11
GPU	Not required — all experiments run CPU-only

2.2 Minimum recommended specification

Component	Minimum	Recommended
Processor	Quad-core (Intel i5 / Ryzen 5 or equivalent)	6+ core (Intel i7/i9, Ryzen 7/9)
RAM	8 GB	16 GB or more (GridSearchCV and raster stacking are memory-intensive)
Storage	20 GB free (SSD preferred)	50+ GB free SSD — raw NetCDF, daily GeoTIFF stacks, and per-year rasters accumulate quickly
GPU	Not required	Not required (XGBoost/CatBoost/LightGBM run CPU-only in this project; a CUDA-capable GPU can optionally accelerate them)
Display	1366×768	1920×1080 or higher (for QGIS raster/map inspection)

Notes:

- GridSearchCV hyperparameter tuning (Random Forest, Extra Trees, AdaBoost) is CPU- and RAM-intensive and can take from seconds to several minutes per model depending on core count.
- Processing 30 years of daily NetCDF rainfall data into per-day GeoTIFFs, yearly rasters, and a 30-year composite is disk I/O and storage intensive; an SSD is strongly recommended over an HDD.

3. Software Requirements

3.1 Operating System

Windows 10/11, Linux (Ubuntu 20.04+), or macOS.

Scripts were developed and tested on Windows 11. All libraries used are cross-platform.

3.2 Core Runtime

Software	Version used / recommended	Purpose
Python	3.10	Core scripting language for the entire pipeline
pip / conda	Latest	Package management (conda recommended for GDAL/geopandas)

3.3 GIS Software

Software	Purpose
QGIS (v3.40 or later recommended)	Raster layer visualization, symbology, and layout for susceptibility, hazard, and risk maps; used for map display and manual raster inspection during processing
GDAL (command-line, incl. gdalwarp)	Reprojection, resampling, and clipping of rainfall and other rasters; called directly via subprocess from the Python scripts

GDAL is a system-level library/binary — install via conda-forge or an OS package manager (not solely via pip) before installing the Python packages below.

3.4 Development Environment

- Visual Studio Code (used for original development) or any Python IDE / editor (Spyder, PyCharm, Jupyter)
- Scripts use "# %%" cell markers compatible with VS Code and Spyder.

3.5 Python Libraries

Library	Purpose
numpy, pandas	Numerical computation and tabular data handling
geopandas, shapely	Vector geospatial data handling
rasterio	Raster I/O, sampling, and reprojection
xarray	Reading and processing NetCDF rainfall data
jenks	Jenks natural breaks classification
scikit-learn	Train/test split, GridSearchCV, evaluation metrics
imbalanced-learn	SMOTE oversampling, Tomek Links undersampling
matplotlib	Plotting (ROC curves, feature importance, comparison charts)
shap	SHAP-based model explainability / feature importance

A consolidated requirements.txt is provided in the project repository.